

Week-6: JDBC CRUD Operations using MySQL Database

Aim

To connect a Java application to a MySQL database using JDBC and perform CRUD (Create, Read, Update, Delete) operations using **PreparedStatement** and **CallableStatement**.

Software Requirements

- WAMP Server (MySQL)
- NetBeans IDE
- Java JDK
- MySQL Connector/J (JDBC Driver)

Theory

JDBC (Java Database Connectivity) is an API that allows Java applications to communicate with relational databases. It provides classes and interfaces to establish a database connection, execute SQL statements, and process query results.

The JDBC architecture consists of:

- JDBC Driver
- DriverManager
- Connection
- Statement
- PreparedStatement
- CallableStatement
- ResultSet

PreparedStatement

PreparedStatement is used to execute parameterized SQL queries. It improves performance and provides security against SQL injection.

CallableStatement

CallableStatement is used to execute stored procedures stored in the database.

CRUD Operations

CRUD represents:

- **Create** – Insert new records into a table
- **Read** – Retrieve records from a table
- **Update** – Modify existing records

- **Delete** – Remove records from a table

Procedure

1. Started WAMP Server and enabled MySQL service.
2. Opened MySQL Console.
3. Created a database named college.
4. Created required tables:
 - Person
 - Employee
 - Product
 - Department
 - Book
5. Created stored procedures:
 - InsertPerson
 - DeleteDepartment
6. Created a Java project in NetBeans.
7. Added MySQL Connector/J driver.
8. Established database connection using JDBC.
9. Developed Java programs using PreparedStatement and CallableStatement.
10. Executed programs and verified results in MySQL.

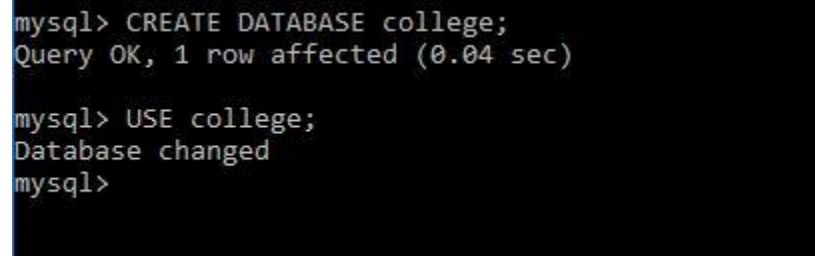
Step 1: Database Creation

SQL Command

CREATE DATABASE college;

USE college;

Screenshot 1: Database Creation



```
mysql> CREATE DATABASE college;
Query OK, 1 row affected (0.04 sec)

mysql> USE college;
Database changed
mysql>
```

Step 2: Creating Tables

Person Table

SAHE-IT

```
CREATE TABLE Person(  
  PersonID INT PRIMARY KEY,  
  FirstName VARCHAR(255),  
  City VARCHAR(255)  
);
```

Employee Table

```
CREATE TABLE Employee(  
  EmpNo INT PRIMARY KEY,  
  EmpName VARCHAR(100),  
  Salary DOUBLE  
);
```

Sample Data:

```
INSERT INTO Employee VALUES  
(101,'Rahul',35000),  
(102,'Ravi',45000),  
(103,'Priya',60000);
```

Product Table

```
CREATE TABLE Product(  
  ProductNo INT PRIMARY KEY,  
  ProductName VARCHAR(100),  
  Price DOUBLE  
);
```

Department Table

```
CREATE TABLE Department(  
  DepartmentID INT PRIMARY KEY,  
  DepartmentName VARCHAR(255),  
  City VARCHAR(255)  
);
```

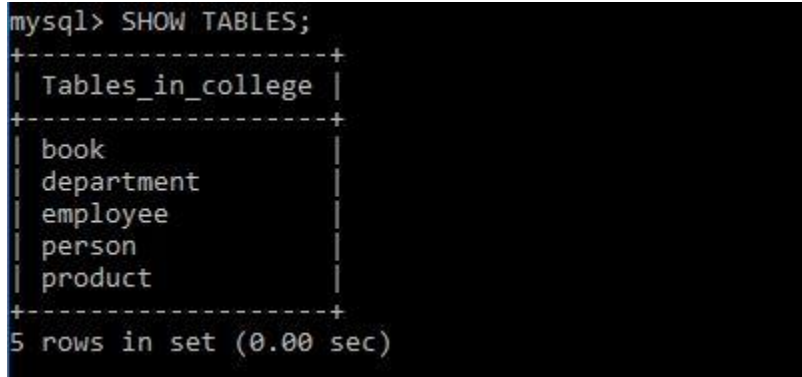
Book Table

```
CREATE TABLE Book(  
  BookNo INT PRIMARY KEY,  
  BookName VARCHAR(100),
```

SAHE-IT

Price DOUBLE
);

Screenshot 2: Tables Created



```
mysql> SHOW TABLES;
+-----+
| Tables_in_college |
+-----+
| book               |
| department         |
| employee           |
| person             |
| product            |
+-----+
5 rows in set (0.00 sec)
```

Step 3: Creating Stored Procedures

Insert Person Procedure

DELIMITER \$\$

```
CREATE PROCEDURE InsertPerson(
IN pid INT,
IN pname VARCHAR(255),
IN pcity VARCHAR(255))
BEGIN
INSERT INTO Person VALUES(pid,pname,pcity);
END$$
```

DELIMITER ;

Delete Department Procedure

DELIMITER \$\$

```
CREATE PROCEDURE DeleteDepartment(IN did INT)
BEGIN
DELETE FROM Department
WHERE DepartmentID=did;
END$$
```

DELIMITER ;

Screenshot 3: Stored Procedures

```
mysql> SHOW PROCEDURE STATUS WHERE Db='college';
```

Db	Name	Type	Definer	Modified	Created	Security_type	Comment
college	DeleteDepartment	PROCEDURE	root@localhost	2026-08-03 21:38:06	2026-08-03 21:38:06	DEFINER	
college	InsertPerson	PROCEDURE	root@localhost	2026-08-03 21:37:39	2026-08-03 21:37:39	DEFINER	

```
2 rows in set (0.03 sec)

mysql>
```

Task 1: Insert Data into Person Table using CallableStatement**Java Program: InsertPerson.java**

```
import java.sql.*;

public class InsertPerson {

    public static void main(String args[]) {

        try {

            Connection con = DBConnection.getConnection();

            CallableStatement cs =
            con.prepareCall("{call InsertPerson(?,?,?)}");

            cs.setInt(1,101);
            cs.setString(2,"Rahul");
            cs.setString(3,"Hyderabad");

            cs.execute();

            System.out.println(
            "Record Inserted Successfully");

            con.close();

        }

        catch(Exception e){
            System.out.println(e);
        }

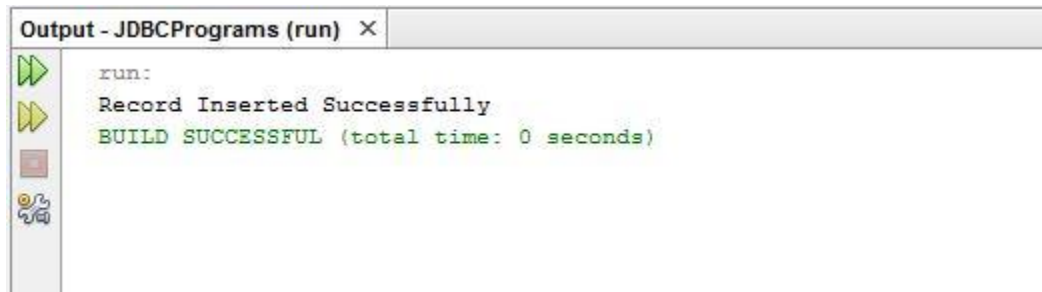
    }

}
```

Output

Record Inserted Successfully

Screenshot 4A: NetBeans Record Insertion



Verification

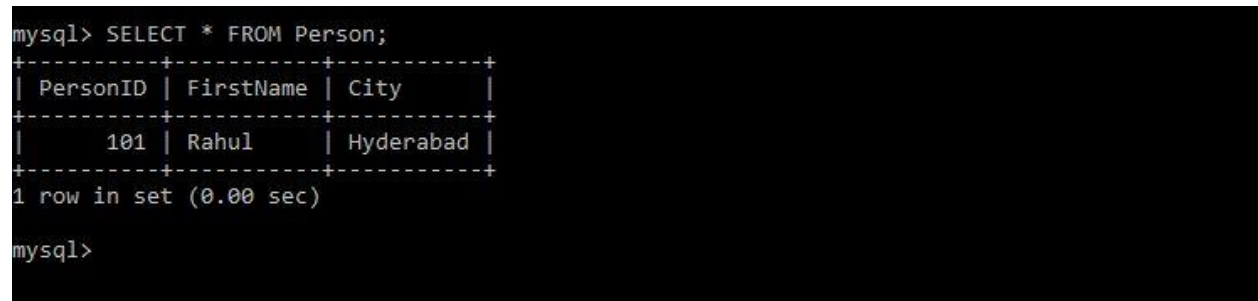
SQL:

```
SELECT * FROM Person;
```

Output:

```
101 Rahul Hyderabad
```

Screenshot 4B: SQL Record Verification



Task 2: Select Employee whose Salary is Greater than X using PreparedStatement

Java Program: SelectEmployee.java

```
import java.sql.*;
import java.util.Scanner;

public class SelectEmployee {
```

SAHE-IT

```
public static void main(String args[]) {  
    try {  
        Connection con = DBConnection.getConnection();  
        Scanner sc=new Scanner(System.in);  
        System.out.print("Enter Salary:");  
        double x=sc.nextDouble();  
        PreparedStatement ps =  
        con.prepareStatement(  
        "SELECT * FROM Employee WHERE Salary > ?");  
        ps.setDouble(1,x);  
        ResultSet rs=ps.executeQuery();  
        while(rs.next()) {  
            System.out.println(  
            rs.getInt(1)+" "+  
            rs.getString(2)+" "+  
            rs.getDouble(3));  
        }  
        con.close();  
    }  
    catch(Exception e){  
        System.out.println(e);  
    }  
}
```

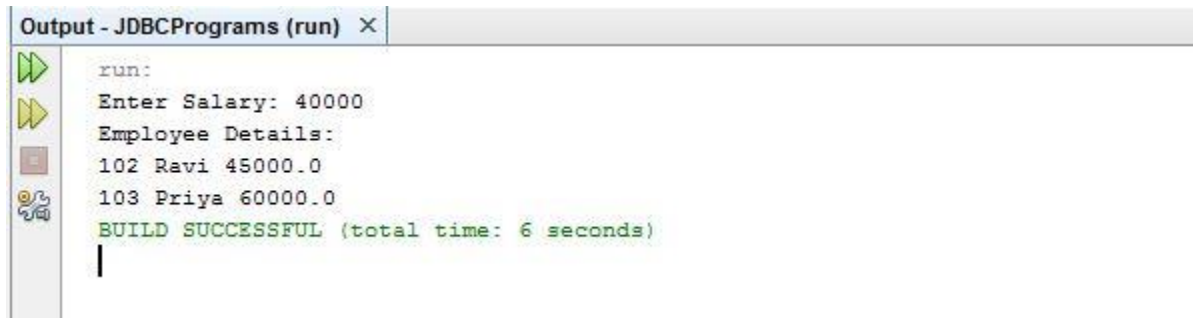
Output

For salary value 40000:

102 Ravi 45000

103 Priya 60000

Screenshot 5A: NetBeans Employee Selection

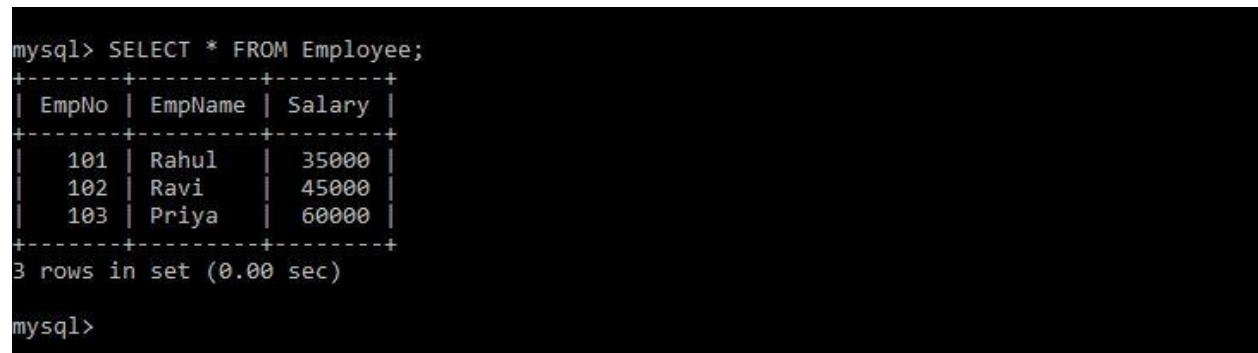


```
run:
Enter Salary: 40000
Employee Details:
102 Ravi 45000.0
103 Priya 60000.0
BUILD SUCCESSFUL (total time: 6 seconds)
```

SQL Verification

```
SELECT * FROM Employee;
```

Screenshot 5B: Employee Table



```
mysql> SELECT * FROM Employee;
+-----+-----+-----+
| EmpNo | EmpName | Salary |
+-----+-----+-----+
| 101   | Rahul   | 35000  |
| 102   | Ravi    | 45000  |
| 103   | Priya   | 60000  |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
```

Task 3: Insert and Display Product Details using PreparedStatement

Java Program: ProductDemo.java

```
import java.sql.*;

public class ProductDemo {

    public static void main(String args[]) {

        try {

            Connection con =
                DBConnection.getConnection();

            PreparedStatement ps =
```


SAHE-IT

```
con.prepareStatement(  
"INSERT INTO Product VALUES(?,?,?)");
```

```
ps.setInt(1,101);  
ps.setString(2,"Laptop");  
ps.setDouble(3,65000);
```

```
ps.executeUpdate();
```

```
System.out.println(  
"Product Inserted Successfully");
```

```
PreparedStatement ps2 =  
con.prepareStatement(  
"SELECT * FROM Product");
```

```
ResultSet rs=ps2.executeQuery();
```

```
while(rs.next()) {
```

```
System.out.println(  
rs.getInt(1)+" "+  
rs.getString(2)+" "+  
rs.getDouble(3));
```

```
}
```

```
con.close();
```

```
}
```

```
catch(Exception e){  
System.out.println(e);  
}
```

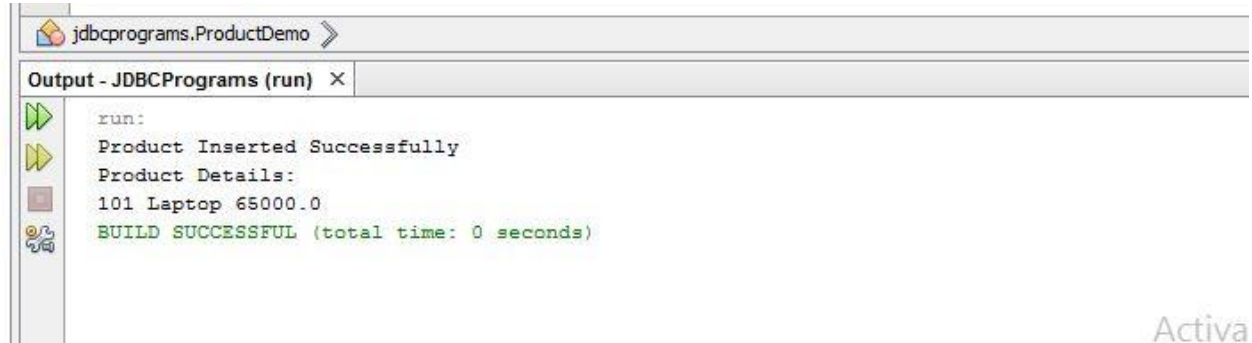
```
}
```

```
}
```

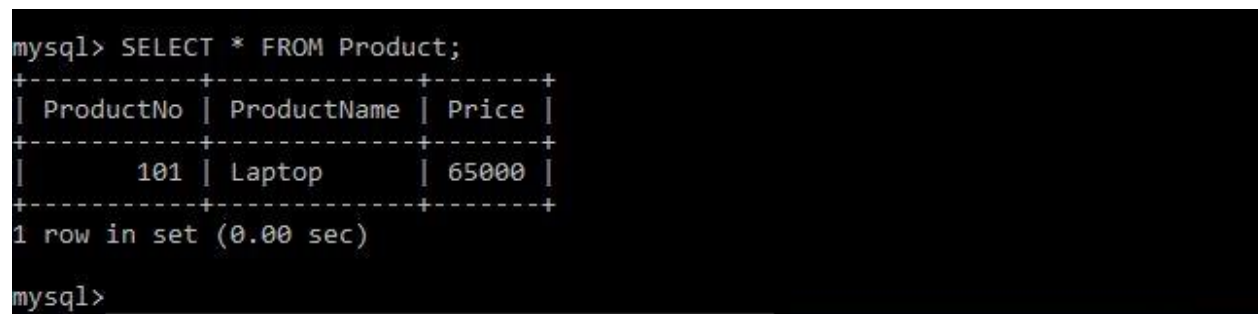
Output

Product Inserted Successfully

101 Laptop 65000

Screenshot 6A: Product Insertion Output**SQL Verification**

```
SELECT * FROM Product;
```

Screenshot 6B: Product Table Verification**Task 4: Delete Department using CallableStatement****Java Program: DeleteDepartment.java**

```
import java.sql.*;

public class DeleteDepartment {

    public static void main(String args[]) {

        try {

            Connection con =
                DBConnection.getConnection();

            CallableStatement cs =
                con.prepareCall(
```

SAHE-IT

```
"{call DeleteDepartment(?)})";

cs.setInt(1,1);

cs.execute();

System.out.println(
"Department Deleted Successfully");

con.close();

}

catch(Exception e){
System.out.println(e);
}

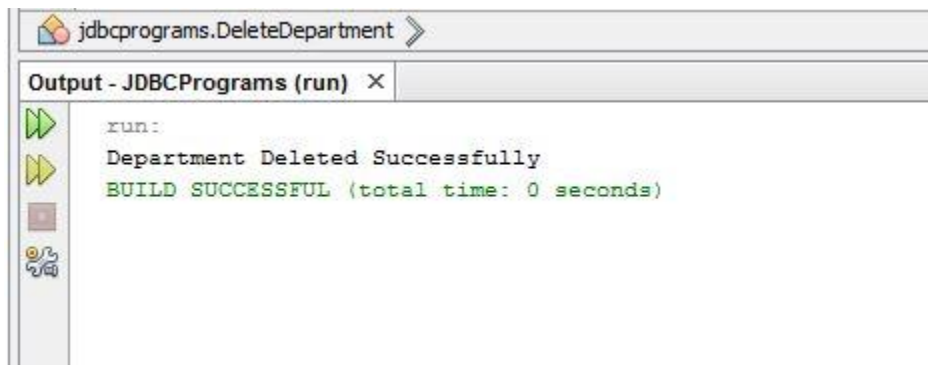
}

}
```

Output

Department Deleted Successfully

Screenshot 7A: Department Deleted



SQL Verification

```
SELECT * FROM Department;
```

After deletion:

2 Mechanical Chennai
3 Electrical Bangalore

Screenshot 7B: MySQL Verification

```
mysql> SELECT * FROM Department;
+-----+-----+-----+
| DepartmentID | DepartmentName | City |
+-----+-----+-----+
| 1 | Computer Science | Hyderabad |
| 2 | Mechanical | Chennai |
| 3 | Electrical | Bangalore |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM Department;
+-----+-----+-----+
| DepartmentID | DepartmentName | City |
+-----+-----+-----+
| 2 | Mechanical | Chennai |
| 3 | Electrical | Bangalore |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

Task 5: CRUD Operations on Book Table

Java Program: BookCRUD.java

Operations performed:

Create

Inserted a book record.

Read

Displayed book records.

Update

Updated book price.

Delete

Deleted the book record.

Output

SAHE-IT

Insert Successful

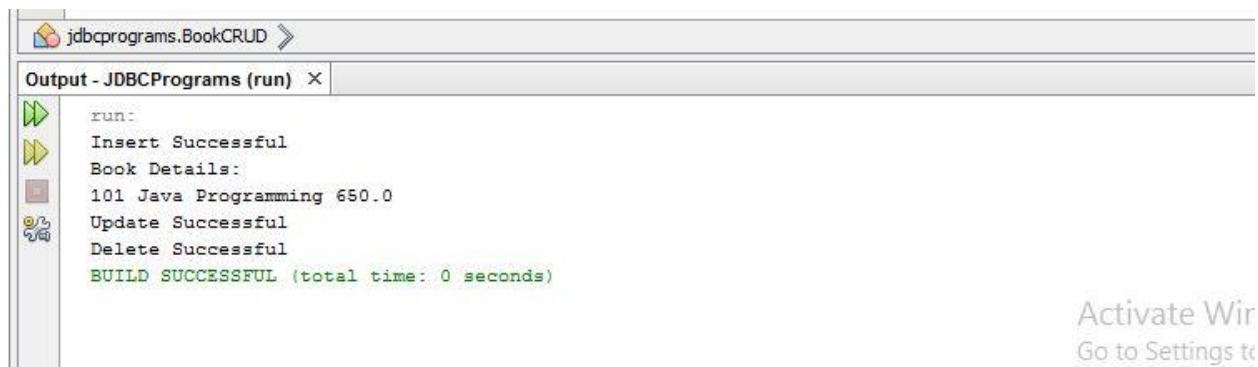
Book Details:

101 Java Programming 650

Update Successful

Delete Successful

Screenshot 8: CRUD Operations Output



Result

The Java application was successfully connected to the MySQL database using JDBC.

CRUD operations were successfully performed using:

- PreparedStatement
- CallableStatement

The records were inserted, retrieved, updated, and deleted successfully from the MySQL database.

Conclusion

JDBC provides an efficient way for Java applications to communicate with databases. Using PreparedStatement and CallableStatement, database operations were performed successfully and verified using MySQL.